

Data Mining and Knowledge Engineering

Assignment 1 – Text Classification

Details

Course Code: COMP723

Course Name: Data Mining and Knowledge Engineering

Lecturer: Dr Parma Nand

Assignment: Assignment 1 – Text Classification

Group Members

- Connor Passingham, 22172974
- Francesca Mae Malveda, 20117041
- Yu Gu, 22187243

Date Submitted: 05/11/2025

Number of Words / Pages:

Contributions

Project Item	Completed By
<i>Data_Covid.ipynb</i>	Connor Passingham
<i>Data_Covid_preprocessed.ipynb</i>	Connor Passingham
<i>Data_Emotion.ipynb</i>	Francesca Mae Malveda & Yu Gu
<i>Data_Emotion_preprocessed.ipynb</i>	Francesca Mae Malveda & Yu Gu
<i>Report</i>	Connor Passingham & Yu Gu & Francesca Mae Malveda
<i>Results Packaging</i>	All

Abstract

This report presents the findings of an experiment evaluating the performance of three text classification algorithms: Naïve Bayes (NB), Neural Network (NN), and Support Vector Machine (SVM), on two free-text datasets. The objective was to assess each algorithm's performance on both raw and pre-processed data, while also examining the impact of parameter tuning on model accuracy. The results showed that the Neural Network achieved the highest overall accuracy. On one dataset, it performed best with pre-processed data and L2 regularization applied, whereas on the other, it achieved optimal performance on raw data. These outcomes provided valuable insights into the relationship between data formatting, parameter optimization, and algorithm performance in text classification tasks.

1. Introduction

1.1 Text Classification Overview

Text classification is the process of assigning predefined labels to text for the sake of categorisation. This classification is based on the content of the text, which may include factors such as vocabulary, punctuation, and context. Similar to numerical classification in that various parameters are analysed to provide evidence supporting a specific label, text classification is based on pattern recognition at its core. Once all evidence is gathered, the model produces a final classification output based on the label with the highest likelihood. However, unlike numerical classification, text classification involves additional complexity. Full contextual information is not always available, and subtleties such as tone or implied meaning can be difficult to interpret from written text alone where numerical data is typically easier to quantify and process digitally.

As with numerical classification algorithms, text classification can be performed using a range of algorithms, each with distinct parameters and underlying mechanisms. These algorithms are first trained on a predefined dataset containing text samples and their corresponding labels, enabling the model to learn patterns and relationships. Once trained, the model is tested on unseen data, and its performance is assessed using various evaluation metrics.

1.2 Purpose of Experiments

The purpose of these experiments is to evaluate the functions, effectiveness, and overall accuracy of three distinct text classification algorithms across two different datasets. Through these experiments, the goal is to develop a deeper understanding of how each algorithm processes textual data, learns from examples, and generalizes to unseen instances.

This investigation aims to explore not only which algorithm performs best in terms of accuracy but also why certain models behave differently depending on the dataset and

the level of pre-processing applied. By examining variations in model performance, the experiments seek to highlight the strengths, weaknesses, and sensitivities of each algorithm when faced with different types of data representation.

A key aspect of this research is understanding the impact of algorithmic parameters and how adjustments, such as regularization methods, learning rates, and feature weighting affect performance outcomes. Additionally, pre-processing techniques such as tokenization, stop word removal, and vectorization are analysed to determine how they influence the models' ability to identify meaningful patterns within the text.

Ultimately, the experiments are designed to provide practical insights into how different text classification models can be effectively applied in real-world natural language processing (NLP) scenarios. This contributes to a broader understanding of the relationship between data preparation, model configuration, and predictive performance in machine learning-based text classification.

1.3 Algorithms Used

1.3.1 Naïve Bayes

The Naïve Bayes classification algorithm is based on Bayes' Theorem, which describes the probability of an event occurring given prior knowledge of related events. The term *naïve* refers to the assumption that all features are **independent** of one another, an assumption that simplifies computation but may not fully represent real-world dependencies in language.

In the context of text classification, the Naïve Bayes algorithm evaluates the likelihood that a given text belongs to a particular category based on the frequencies and distribution of words learned during training. Essentially, it determines how probable a label is for a test instance by comparing its word occurrences to those of known, labelled examples in the training dataset.

1.3.2 Neural Network

A Neural Network (NN) is a machine learning algorithm inspired by the structure and function of the human brain. It consists of interconnected nodes, or neurons, organized into layers that process information and identify patterns within data. Each neuron receives input, applies a mathematical transformation through an activation function, and passes the result to the next layer.

In the context of text classification, neural networks learn to recognize relationships between words and patterns in language that correlate with specific labels. Through multiple training iterations, the network adjusts internal weights to minimize classification errors. Neural networks are particularly powerful for complex text data as they can capture non-linear relationships and contextual meaning beyond simple word frequency.

1.3.3 Support Vector Machine

A **Support Vector Machine (SVM)** is a supervised learning algorithm used for both classification and regression tasks. It works by finding the **optimal hyperplane** that best separates data points of different classes in a high-dimensional space. The aim is to maximize the margin between data points of different labels, enhancing generalization to unseen data.

For text classification, SVMs transform text data into numerical feature vectors and then determine the boundary that best distinguishes between categories. SVMs are effective for text data because they handle high-dimensional input well and are resistant to overfitting, especially when the number of features (words) exceeds the number of training samples.

1.3.4 Algorithm Comparison

The three algorithms Naïve Bayes, Neural Network, and Support Vector Machine, were selected to provide a diverse comparison of classification approaches. Naïve Bayes represents a probabilistic model that is simple and efficient for text-based tasks. Neural Networks provide a deep learning approach, capable of capturing complex, non-linear relationships in language. In contrast, SVM offers a boundary-based model that performs well on high-dimensional data such as word features.

By evaluating these algorithms on both raw and pre-processed text, we can assess how different learning methods respond to variations in data quality and structure, ultimately determining which is most effective for text classification.

2. Data Description

2.1 Dataset 1 - Covid News Articles

This study uses the Corona_NLP dataset, published on Kaggle by *timlayer*. It contains COVID-related tweets collected during 2020–2022 and is distributed in two ready-to-use CSV files: 41,157 tweets for training and 3,798 for testing. The dataset is openly available for educational analysis.

Each record provides the tweet text and a five-level sentiment label. The key fields used in our pipeline are:

- **OriginalTweet** - the raw tweet content that we model on;
- **Sentiment** - the target class: *Extremely Negative, Negative, Neutral, Positive, or Extremely Positive*.
- Auxiliary metadata (e.g., TweetAt, UserName, Location) that we do not model.

There are a few characteristics worth noting. Tweets are short, informal, and often include emojis, hashtags, and URLs; this introduces noise but also conveys affective

cues. Class frequencies are imbalanced (Neutral and Positive occur more often than the extremes), so we report both micro and macro metrics later. To avoid over-cleaning, we apply minimal, task-aware preprocessing: lowercase, remove URLs/extra whitespace, and keep negations and emoji. Vectorization follows 3.2 (TF-IDF with n-gram settings as specified there). The train/test split provided with the dataset is preserved; hyper-parameters are selected on the training set only, and final scores are reported on the held-out test set.

2.2 Dataset 2 – Emotion Detection

The Emotion Detection dataset was originally collected by data.world approximately five years ago and later made publicly available on Kaggle for research and text classification purposes. It is an open-source dataset released under the public domain, enabling unrestricted academic and experimental use.

The dataset contains 40,000 records stored in a CSV file with three columns:

- **tweet_id** – a unique numerical identifier for each tweet.
- **content** – the main tweet text, which includes informal elements such as slang, punctuation, emojis, and varied sentence structures.
- **sentiment** – the target label representing the underlying emotion expressed in the tweet. There are 13 emotion categories, including *neutral*, *sadness*, *worry*, *happiness*, *love*, and others.

The dataset’s textual nature and inclusion of real-world social media language make it ideal for testing the performance of text classification algorithms on unstructured data. Some tweets contain noisy or ambiguous expressions, requiring careful preprocessing before model training. Much like the Covid dataset, the use of shorthand language and varying levels of both formality and correct grammar provides a point of interest when testing for classification accuracy. However, the scope of the contents is broader given the lack of a consistent topic between the collected tweets as opposed to Dataset 1’s content all relating to Covid in some form. Furthermore, this dataset is inherently presented in a singular file as opposed to Dataset 1’s files already being separated into separate training and testing sets. Beyond the expected duplicates in the sentiment column as the primary classifier, no duplicate data was detected, nor were there any null values or fields.

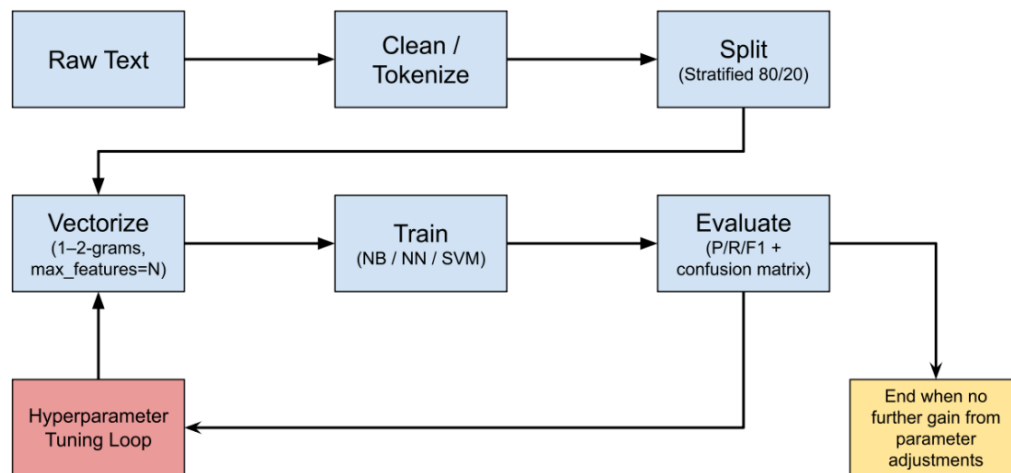
3.0 Methodology

3.1 Data pipeline

Figure 3.1 illustrates the end-to-end workflow used throughout the experiment. The process begins with raw text, which is cleaned and tokenized before being split into stratified training and testing subsets (80/20). Text data is then vectorized using TF-IDF to convert it into numerical features suitable for model training. Three algorithms: Naïve Bayes (NB), Neural Network (NN), and Support Vector Machine (SVM), were trained and

evaluated using precision, recall, F1-score, and confusion matrices. A hyperparameter tuning loop was applied to refine model performance, with iterations continuing until no further gains were achieved

Figure 3.1 Data Pipeline



3.2 Pre-Processing Steps

A lightweight pre-processing pipeline was implemented to improve data quality and model performance while avoiding unnecessary complexity.

- **Lowercasing:** All text was converted to lowercase to ensure consistency and prevent the same word in different cases (e.g., “Covid” vs “COVID”) from being treated as separate features. This helps reduce feature sparsity and improves generalization.
- **Tokenisation:** Sentences were split into individual words or tokens, allowing the model to analyse text at the word level. This is essential for building TF–IDF feature representations that capture the relative importance of each token.
- **Stop-word Removal:** Common function words such as “and,” “the,” and “is” were selectively removed. These words typically add little semantic value and can dominate frequency-based features, potentially masking more meaningful patterns in the data.
- **URL and Symbol Clean-up:** Non-informative elements such as web links, hashtags, and special symbols were removed to prevent them from skewing the model’s understanding of the text. This reduces noise and ensures that only relevant textual information contributes to feature generation.
- **Consistent Pipeline Across Models:** The same pre-processing procedure was applied to all algorithms to maintain fairness in comparison. Additionally, the TF–IDF vectorizer was fitted on the training set only, preventing data leakage and ensuring realistic evaluation of model performance.

3.3 Metrics

This experiment used a range of evaluation metrics to assess the accuracy and overall performance of each classification algorithm. The following key metrics were applied:

- **Precision:** Measures the proportion of correctly predicted positive instances out of all predicted positives. It indicates how well the model avoids false positives.
- **Recall:** Measures the proportion of actual positive instances that were correctly identified. It reflects the model's ability to capture all relevant cases.
- **F1-Score:** Represents the harmonic mean of precision and recall, providing a balanced measure that is particularly useful when dealing with class imbalances.
- **Accuracy:** Calculates the overall proportion of correctly classified samples across all classes, giving a general sense of model performance.

For consistency across models, macro averaging was used when calculating Precision, Recall, and F1-Score. This approach computes the metric independently for each class and then takes the unweighted mean, ensuring that all sentiment categories contribute equally to the final score regardless of class frequency. Using macro averages prevents dominant classes from biasing the overall performance results, allowing for a fairer comparison of model effectiveness across all sentiment levels.

In addition to these metrics, **confusion matrices** were used to visually evaluate the classification outcomes. These matrices display the distribution of correct and incorrect predictions across sentiment classes, helping to identify where each model performed well or struggled. The full confusion matrices are included in the appendix for reference.

4.0 Experiments

A series of experiments were conducted to fine-tune and evaluate three classification algorithms: Naïve Bayes, Neural Network, and Support Vector Machine (SVM), using the same TF-IDF feature representations and train/test split. Each experiment followed a consistent process as outlined in Section 5: data was cleaned, vectorised, split (80/20 stratified), and then trained using distinct parameter settings per model. The aim was to systematically assess how hyperparameter tuning and architectural choices affected model accuracy, stability, and generalisation.

Naïve Bayes

- A **grid search** was used to identify the optimal alpha (smoothing) parameter, testing a range of small float values (e.g., 0.1–2.0).

- Additive smoothing prevents zero probabilities for words not encountered during training, ensuring the model remains stable when processing unseen vocabulary.
- Each configuration was evaluated using the same pre-processed TF-IDF inputs and scored with precision, recall, F1, and accuracy metrics.
- This experiment was designed to determine the optimal balance between overfitting (low alpha) and underfitting (high alpha), providing a reproducible tuning process that can be replicated by rerunning the same parameter grid.

Neural Network

- A dense feedforward Neural Network architecture was implemented using a single hidden layer, ReLU activation, and a softmax output layer for multi-class classification.
- **Learning rate** experiments were conducted by testing progressively smaller values (e.g., 0.01, 0.001, 0.0005) to observe convergence behaviour and minimise oscillation during training.
- **Batch Normalization** layers were introduced after the dense layer to normalise internal activations, improving training stability and allowing faster convergence.
- **L2 Regularization** was added to the network's weights to discourage large coefficient values, mitigating overfitting in high-dimensional TF-IDF input space.
- Each model was trained over multiple epochs with consistent batch sizes and early stopping to prevent overtraining. These controlled variations allowed us to isolate the effects of learning rate, normalization, and regularisation on generalisation performance.

Support Vector Machine (SVM)

- A **LinearSVC** model was used, given the high dimensionality and sparse nature of TF-IDF features.
- A **grid search** was performed over a range of **C values** (e.g., 0.1–10) to determine the optimal regularisation strength, balancing model flexibility and overfitting risk.
- The **maximum iteration limit** was increased (from the default to 10,000) to allow the model to converge on large feature sets.
- **Dual optimisation** was disabled (dual=False) to enhance computational efficiency, as the number of samples exceeded the number of features.
- Each model configuration was evaluated using identical train/test partitions and the same TF-IDF representations, ensuring reproducibility.

All experiments were run under consistent conditions, using the same random state (42) for reproducibility and identical preprocessing pipelines. Results were compared using precision, recall, F1-score, accuracy, and confusion matrices, allowing both quantitative and visual assessment of performance differences. This structured approach ensures that the experimental setup can be replicated in full by following the

described steps and parameters.

5.0 Results and Discussion

5.1 Links to Code

Dataset_Covid_Raw:

<https://colab.research.google.com/drive/1qdieEJeqAJPVJpX5tV8w3dDLMTb08Rdy?usp=sharing>

Dataset_Covid_Pre-Processed:

<https://colab.research.google.com/drive/1HFb7X3FtYmgHoLNI2t-p6HKvBzzeibg4>

Dataset_Emotion_Raw:

https://colab.research.google.com/drive/10o7aNvHuYDZuE8F5BGYPvC2c9cy6f8_5?usp=sharing

Dataset_Emotion_Pre-Processed:

https://colab.research.google.com/drive/1ClgX1lEOED-OWffPyTaZ_25rVKTTfmHb?usp=sharing

5.2 Results Tables

Table 5-1. Covid (Corona_NLP) - Raw Data

Model/Variant	Overall Accuracy	Macro AVG Precision	Macro AVG Recall	Macro AVG F1
MultinomialNB (baseline)	0.44	0.59	0.39	0.39
MultinomialNB ($\alpha=0.1$, tuned)	0.46	0.57	0.42	0.43
LinearSVM (baseline, C=1)	0.57	0.57	0.58	0.57
LinearSVM (tuned: C=1,	0.56	0.57	0.58	0.57

dual=False, max_iter)				
Neural Net (baseline)	0.64	0.65	0.65	0.64
Neural Net + L2(best)	0.66	0.70	0.65	0.67

Table 5-2. Covid (Corona_NLP) - Pre-processed Data

Model/Variant	Overall Accuracy	Macro AVG Precision	Macro AVG Recall	Macro AVG F1
Neural Net + L2(best)	0.56	0.56	0.55	0.56
MultinomialNB ($\alpha=0.1$, tuned)	0.45	0.52	0.43	0.45
LinearSVM (baseline, C=1)	0.52	0.54	0.54	0.54

Table 5-3. Emotion (tweet_emotions) - Raw Data

Model/Variant	Overall Accuracy	Macro AVG Precision	Macro AVG Recall	Macro AVG F1
MultinomialNB (baseline)	0.32	0.22	0.14	0.12
MultinomialNB ($\alpha=0.5$, tuned)	0.32	0.18	0.14	0.13
LinearSVM (baseline, C=1)	0.33	0.22	0.18	0.19
LinearSVM (tuned: C=0.1, dual=False, max_iter)	0.36	0.24	0.24	0.18

Neural Net (baseline)	0.33	0.19	0.18	0.17
Neural Net + Learning Rate Adjustment	0.35	0.18	0.17	0.16

Table 5-4. Emotion (tweet_emotions) - Pre-processed Data

Model/Variant	Overall Accuracy	Macro AVG Precision	Macro AVG Recall	Macro AVG F1
MultinomialNB (baseline)	0.33	0.26	0.15	0.14
LinearSVM (tuned: C=0.1, dual=False, max_iter)	0.36	0.25	0.18	0.18
Neural Net + Learning Rate Adjustment (best)	0.26	0.16	0.16	0.16

5.2 Analysis and Comparisons

Across the two datasets, clear performance differences emerged between the three classification algorithms. Overall, the Neural Network (NN) consistently outperformed both Naïve Bayes (NB) and Support Vector Machine (SVM), particularly on the *Covid (Corona_NLP)* dataset. The Neural Network with L2 regularisation achieved the highest accuracy of 0.66 on raw data, accompanied by strong macro-averaged precision (0.70) and recall (0.65). This suggests that the NN was effective at learning complex nonlinear patterns present in the textual data, even without extensive preprocessing.

The SVM demonstrated robust and consistent results across both datasets, maintaining accuracy scores around 0.52-0.57 on the Covid data and 0.36 on the Emotion dataset. Its performance stability, even with minimal hyperparameter tuning, highlights its ability to generalise effectively. Meanwhile, Naïve Bayes performed

weakest overall, with accuracies ranging from 0.32 to 0.46, reflecting its simplicity and sensitivity to correlated features in TF-IDF vectors.

A notable pattern was the decline in performance following pre-processing. While pre-processing was expected to improve data quality and reduce noise, all three models performed slightly worse after cleaning and tokenisation. This was particularly evident in the Covid dataset, where Neural Network accuracy dropped from 0.66 to 0.56, implying that the removal of certain linguistic cues (such as punctuation, casing, or emojis) reduced the models' ability to distinguish sentiment nuances.

In contrast, the Emotion dataset produced lower and more uniform results across all models, with limited improvement from tuning. This suggests a higher level of dataset noise and class ambiguity, which constrained the models' discriminative power. Overall, while SVM remained stable, the Neural Network proved most adaptable, achieving top results when regularisation was applied to control overfitting.

5.3 Observations and Anomalies

Several key observations and anomalies were identified from the experimental outcomes.

- **Neural Network superiority on raw data:** The best overall performance (accuracy = 0.66) was achieved by the Neural Network with L2 regularisation on the raw Covid dataset. This indicates that the network effectively leveraged complex contextual patterns that were diluted after pre-processing.
- **Effect of pre-processing:** Contrary to expectations, pre-processing did not consistently improve results. For both datasets, accuracies dropped after text cleaning, suggesting that elements such as emojis, capitalisation, and punctuation provided useful sentiment cues that were lost during normalisation.
- **SVM consistency:** The SVM exhibited the most **stable results** across datasets, achieving similar accuracies on both raw and pre-processed data (Covid: ~0.52–0.57; Emotion: ~0.36). This reinforces the strength of linear decision boundaries in handling sparse, high-dimensional text features.
- **Naïve Bayes sensitivity:** NB models struggled most with unbalanced classes and feature correlations, achieving lower macro-average recall and F1 scores. Even with alpha tuning, performance gains were minimal, highlighting its limitations on complex linguistic data.
- **Macro-average metric patterns:** Macro-averaged precision, recall, and F1-scores were consistently lower than overall accuracy across all experiments. This discrepancy reflects **class imbalance**, where dominant sentiment classes were predicted more accurately than minority ones.
- **Dataset variability:** The Emotion dataset produced uniformly lower scores across all models, likely due to its informal language and inconsistent emotional labelling. This variability limited the effectiveness of parameter tuning and model optimisation.

In summary, these observations underline that while neural architectures benefit from regularisation and raw data richness, SVMs remain reliable across diverse datasets. The results also emphasise the importance of cautious pre-processing, as excessive cleaning can inadvertently remove contextually important features.

6. Conclusion and Reflections

6.1 Summary of Findings

The experiments conducted in this study provided valuable insights into the performance and behaviour of three key text classification algorithms, Naïve Bayes (NB), Support Vector Machine (SVM), and Neural Network (NN), across two distinct datasets: *Covid (Corona_NLP)* and *Emotion (tweet_emotions)*.

Across all experiments, the Neural Network emerged as the best-performing model, achieving a top accuracy of 0.66 on the raw Covid dataset when combined with L2 regularisation. This model also demonstrated strong macro-averaged precision and recall scores, indicating that regularisation effectively balanced generalisation and overfitting. In contrast, both SVM and Naïve Bayes achieved moderate results, with SVM showing stable yet less dynamic performance and Naïve Bayes struggling to adapt to complex or unbalanced data distributions.

A consistent finding was that pre-processing did not always enhance model performance. Although text cleaning and tokenisation are typically expected to improve feature consistency, the results indicated that removing certain linguistic and contextual elements, such as punctuation, capitalisation, and emojis reduced sentiment-related cues. As a result, models trained on raw data often achieved higher accuracy, particularly in the Covid dataset.

The Emotion dataset presented additional challenges due to its noisy and informal nature, resulting in generally lower scores across all algorithms. This suggests that dataset complexity, label ambiguity, and varying writing styles significantly affect model learning and predictive accuracy.

Overall, the findings highlight that Neural Networks with appropriate parameter tuning (such as learning rate adjustments and L2 regularisation) can outperform traditional algorithms in text classification tasks. However, the effectiveness of pre-processing and tuning depends strongly on dataset structure, feature richness, and the presence of balanced class distributions.

6.2 Reflections and Future Work

Overall, this report was successful in enhancing our understanding of text classification and the effectiveness of various machine learning algorithms. Through this project, we deepened our knowledge of the Naïve Bayes, Neural Network, and Support Vector Machine (SVM) algorithms, exploring how each responds to different parameter

adjustments and data processing methods. The experiments provided valuable insight into how text-based data can be transformed, represented, and classified to produce meaningful results.

By implementing several pre-processing steps, including tokenization, stopwords removal, and vectorization, we were able to observe first-hand how these stages influence model performance. This practical experience highlighted the importance of data preparation in text classification tasks and demonstrated that pre-processing can both enhance and, in some cases, reduce model accuracy depending on the dataset characteristics.

If we were to implement this project again, we would consider testing additional datasets with more controlled or topic-specific content. The use of Twitter data introduced a significant level of linguistic variation and noise, as tweets often contain slang, abbreviations, and irrelevant information. Using a dataset with more consistent language or structured content could help isolate the effects of specific model adjustments more clearly and provide a more stable basis for comparison.

Another aspect we would look to improve in future work is the depth of parameter tuning and optimization. While grid search and L2 regularization were effective in identifying strong baseline settings, further refinements could be achieved through methods such as cross-validation, learning rate scheduling, momentum tuning, and layer-specific optimization within the neural network. Exploring different kernel functions and gamma values for SVM, or alternative smoothing techniques for Naïve Bayes, could also provide a more comprehensive understanding of how subtle parameter changes influence model behaviour.

Overall, we believe our experiments were conducted thoroughly, and the resulting analysis was both conclusive and reliable. The findings provided a comprehensive overview of how each algorithm performs under different experimental conditions, strengthening our understanding of the interplay between data preparation, parameter adjustment, and model performance.

Appendix

Confusion Matrices

Fig. 1 — Covid/NN (best) confusion matrix (test).

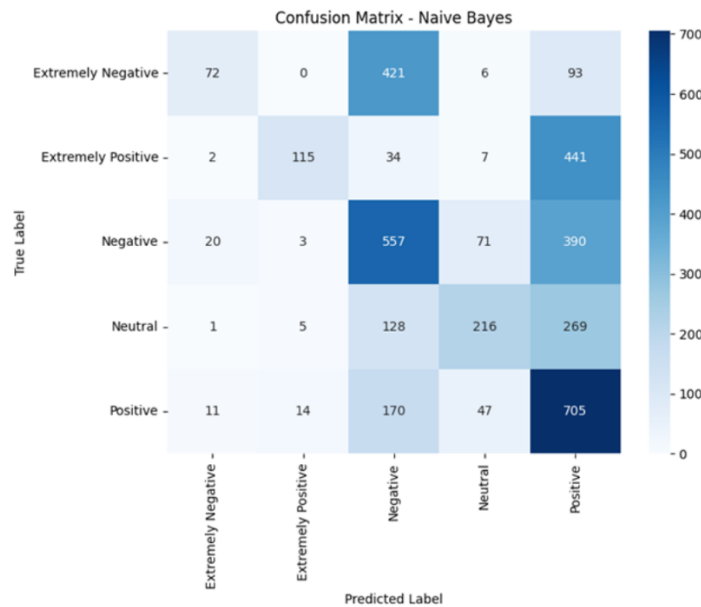


Fig. 2 — Covid/LinearSVC confusion matrix (test).

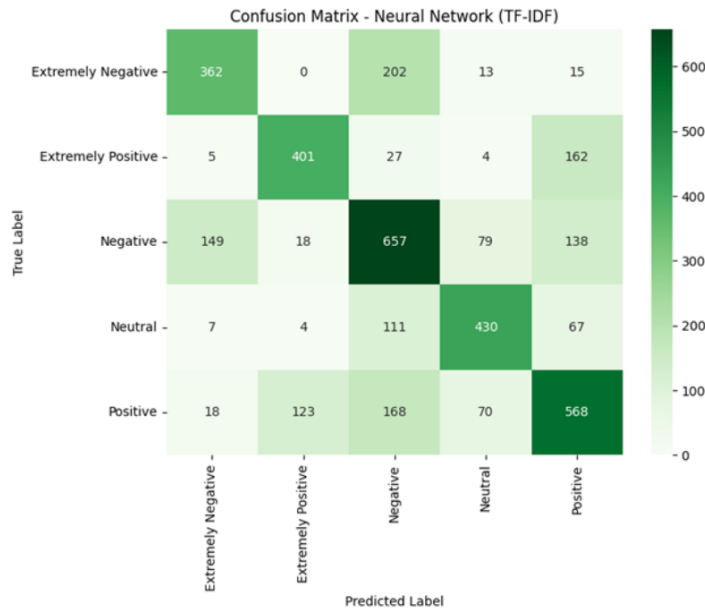
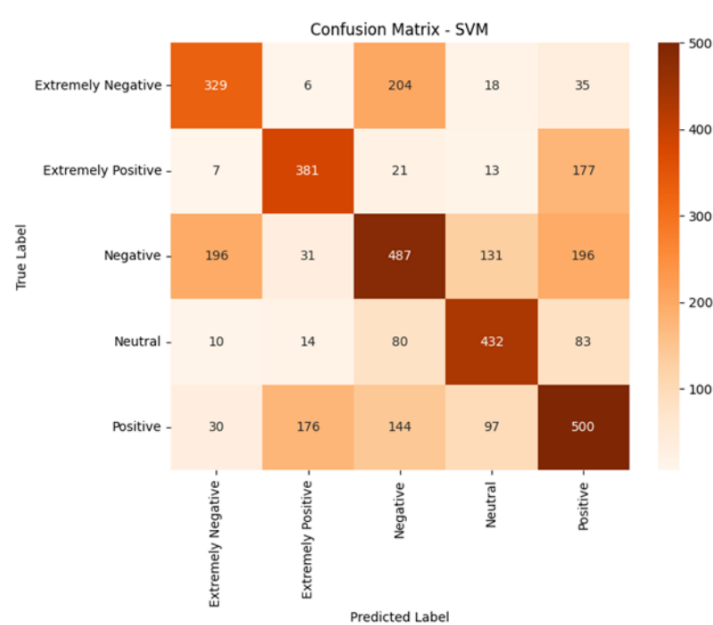


Fig. 3 — Covid/Naïve Bayes confusion matrix (test).



Classification Reports

Raw Data Reports

Fig 6. SVM Raw Data

SVM Model Performance:
Accuracy: 0.5606

Classification Report:

	precision	recall	f1-score	support
Extremely Negative	0.58	0.56	0.57	592
Extremely Positive	0.63	0.64	0.63	599
Negative	0.52	0.47	0.49	1041
Neutral	0.63	0.70	0.66	619
Positive	0.50	0.53	0.52	947
accuracy			0.56	3798
macro avg	0.57	0.58	0.57	3798
weighted avg	0.56	0.56	0.56	3798

Fig 7. NN Raw Data

Neural Network Performance:
Accuracy: 0.6388

Classification Report:					
	precision	recall	f1-score	support	
Extremely Negative	0.66	0.63	0.64	592	
Extremely Positive	0.72	0.70	0.71	599	
Negative	0.59	0.58	0.58	1041	
Neutral	0.72	0.69	0.71	619	
Positive	0.59	0.63	0.61	947	
accuracy			0.64	3798	
macro avg	0.65	0.65	0.65	3798	
weighted avg	0.64	0.64	0.64	3798	

Fig 8. NB Raw Data

Naive Bayes Performance:
Accuracy: 0.4384

Classification Report:					
	precision	recall	f1-score	support	
Extremely Negative	0.68	0.12	0.21	592	
Extremely Positive	0.84	0.19	0.31	599	
Negative	0.43	0.54	0.47	1041	
Neutral	0.62	0.35	0.45	619	
Positive	0.37	0.74	0.50	947	
accuracy			0.44	3798	
macro avg	0.59	0.39	0.39	3798	
weighted avg	0.55	0.44	0.41	3798	

Parameter Adjustment Reports

Fig 9. NB Best Adjustment

--- Tuned Naive Bayes Performance ---
Best alpha: 0.1
Accuracy: 0.4616

Classification Report:					
	precision	recall	f1-score	support	
Extremely Negative	0.67	0.21	0.32	592	
Extremely Positive	0.74	0.24	0.36	599	
Negative	0.44	0.55	0.49	1041	
Neutral	0.65	0.41	0.50	619	
Positive	0.38	0.70	0.49	947	
accuracy			0.46	3798	
macro avg	0.57	0.42	0.43	3798	
weighted avg	0.54	0.46	0.45	3798	

Fig 10. SVM Best Adjustment

--- Improved SVM (Higher max_iter, dual=False) ---

Accuracy: 0.5606

	precision	recall	f1-score	support
Extremely Negative	0.58	0.56	0.57	592
Extremely Positive	0.63	0.64	0.63	599
Negative	0.52	0.47	0.49	1041
Neutral	0.63	0.70	0.66	619
Positive	0.50	0.53	0.52	947
accuracy			0.56	3798
macro avg	0.57	0.58	0.57	3798
weighted avg	0.56	0.56	0.56	3798

Fig 11. NN Best Adjustment

--- Neural Network with L2 Regularization ---

Accuracy: 0.6590

	precision	recall	f1-score	support
Extremely Negative	0.77	0.56	0.65	592
Extremely Positive	0.80	0.61	0.69	599
Negative	0.61	0.64	0.62	1041
Neutral	0.75	0.73	0.74	619
Positive	0.57	0.73	0.64	947
accuracy			0.66	3798
macro avg	0.70	0.65	0.67	3798
weighted avg	0.68	0.66	0.66	3798

Pre-processed Reports

Fig 11. NB Best Adjustment (Pre-Processed)

Multinomial Naïve Bayes Results (alpha=0.1):

Accuracy: 0.454449710373881

	precision	recall	f1-score	support
Extremely Negative	0.57	0.27	0.37	592
Extremely Positive	0.65	0.31	0.42	599
Negative	0.41	0.51	0.46	1041
Neutral	0.58	0.48	0.53	619
Positive	0.38	0.59	0.46	947
accuracy			0.45	3798
macro avg	0.52	0.43	0.45	3798
weighted avg	0.49	0.45	0.45	3798

Fig 11. SVM Best Adjustment (Pre-Processed)

Support Vector Machine Results (C=1.0):

Accuracy: 0.5239599789362822

	precision	recall	f1-score	support
Extremely Negative	0.57	0.54	0.55	592
Extremely Positive	0.61	0.59	0.60	599
Negative	0.48	0.43	0.45	1041
Neutral	0.56	0.64	0.60	619
Positive	0.46	0.50	0.48	947
accuracy			0.52	3798
macro avg	0.54	0.54	0.54	3798
weighted avg	0.52	0.52	0.52	3798

Fig 11. NN Best Adjustment (Pre-Processed)

Neural Network (MLP) Results (tuned):

Accuracy: 0.5468667719852554

	precision	recall	f1-score	support
Extremely Negative	0.60	0.49	0.54	592
Extremely Positive	0.65	0.58	0.61	599
Negative	0.49	0.53	0.51	1041
Neutral	0.58	0.63	0.60	619
Positive	0.51	0.53	0.52	947
accuracy			0.55	3798
macro avg	0.56	0.55	0.56	3798
weighted avg	0.55	0.55	0.55	3798